Modernizing Office 365 Connector Workflows to Microsoft Graph (Logic Apps / Power Automate)

Implementation Solution Guide

Version 1.0.0 · 2026-07-17

Contents

Executive Summary — Modernizing Office 365 Connector Workflows to Microsoft Graph (Logic Apps / Power Automate) 4

What you get . . . 4

What this delivers . . . 4

Honest scope . . . 4

The standard deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Modernizing Office 365 Connector Workflows to Microsoft Graph 5

Why this is a program, not a one-off . . . 5

The inventory → classify → migrate → govern arc . . . 5

The per-flow playbook leans on the sibling products (reference, don't re-teach) . . . 6

Logic Apps and Power Automate are not the same — treat the difference honestly . . . 6

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 8

2. Requirements Guide — Modernizing Office 365 Connector Workflows to Microsoft Graph 8

2.1 Read access to enumerate the Logic Apps estate . . . 8

2.2 Read access to enumerate Power Automate flows (Power Platform) . . . 8

2.3 Rights to create managed identities and grant application permissions . . . 9

2.4 Rights to scope mailbox access (defer the mechanism to sibling #4) . . . 9

2.5 Microsoft.Graph PowerShell and an enumeration method for Power Automate . . . 9

2.6 A named pilot cohort . . . 10

2.7 Change control, parity baseline, and per-flow rollback . . . 10

2.8 National-cloud endpoint confirmation (if applicable) . . . 10

Checklist . . . 10

Global Requirements — Azure Solutions 11

G1. A Microsoft 365 / Entra tenant . . . 11

G2. Administrative roles (least privilege) . . . 11

G3. Licensing baseline . . . 11

G4. Network connectivity to Microsoft 365 / Azure endpoints . . . 12

G5. Change control & maintenance window . . . 12

G6. Backup / rollback posture . . . 12

3. Implementation Guide — Modernizing Office 365 Connector Workflows to Microsoft Graph 12

Naming (set once, reuse verbatim) . . . 13

Phase 1 — Inventory the connector estate and its authentication . . . 13

Phase 2 — Classify by risk and Graph-migratability . . . 14
Phase 3 — Configure the pilot cohort's managed-identity Graph targets . . . 14
Phase 4 — Pilot the migration . . . 15
Phase 5 — Validate the pilot (the gate) . . . 15
Phase 6 — Expand in waves . . . 16
Phase 7 — Govern the end state (prevent regression) . . . 16
5. Testing Guide — Modernizing Office 365 Connector Workflows to Microsoft Graph 17
Test 1 — Inventory completeness . . . 17
Test 2 — Per-flow parity (migrated matches legacy) . . . 17
Test 3 — Migrated flow uses managed identity + Graph . . . 17
Test 4 — No residual stored credentials in the migrated cohort . . . 17
Test 5 — Scoped mailbox permission (least privilege, per #4) . . . 17
Test 6 — Per-flow rollback works . . . 18
Test 7 — Program progress / no regression . . . 18
Pass/fail gate . . . 18
6. Troubleshooting Guide — Modernizing Office 365 Connector Workflows to Microsoft Graph 18
A. A flow was missed in the inventory . . . 18
B. Parity break after cutover . . . 19
C. Power Automate flow can't use a managed identity . . . 20
D. Permission / consent gaps . . . 20
E. Regression to connector-based auth . . . 21
F. Diagnostics quick reference . . . 21
G. When to escalate . . . 21
7. Best Practices — Modernizing Office 365 Connector Workflows to Microsoft Graph 21
8. Deliverables Summary — Modernizing Office 365 Connector Workflows to Microsoft Graph 22
Documentation . . . 22
PowerShell scripts . . . 23
What the buyer receives . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Modernizing Office 365 Connector Workflows to Microsoft Graph (Logic Apps / Power Automate)

White-label · Azure Solutions

One workflow hitting the Office 365 connector's authentication wall is a fix; a whole estate of them is a program. Organizations accumulate Logic Apps and Power Automate flows built on the Office 365 Outlook / Users / Excel connectors — all authenticated as a user or a shared service account, all carrying a standing credential nobody re-authorized. Together they are a large, quiet authentication-risk surface no single fix retires.

This is a complete, **run-it-yourself** program for migrating that estate onto **managed-identity Microsoft Graph** workflows at scale, written for an administrator with no prior migration-program experience.

What you get

- A plain-language **Solution Overview** of why this is a program — the inventory→classify→migrate→govern arc, and the per-flow playbook that leans on the sibling products (#1/#2/#3/#5) and defers mailbox scoping to #4.
- A **Requirements Guide** with verify / implement-if-missing for read access across both platforms, the rights to create managed identities and grant Graph application permissions, and the pilot cohort.
- An **Implementation Guide** that is the program runbook — inventory both platforms, classify by risk and Graph-migratability, configure pilot targets, cut over pilot-first with a go/no-go gate, expand in waves with per-flow rollback, and govern the end state.
- Five **PowerShell scripts** for discovery, classification, PASS/FAIL validation, a program dashboard, and a `-WhatIf`-gated app-role helper.
- **Testing** and **Troubleshooting** guides for a confident, reversible rollout.

What this delivers

A connector estate **inventoried** (Logic Apps + Power Automate, with each flow's auth model), **risk-ranked and classified** by Graph-migratability, **migrated wave by wave** onto managed-identity Graph using a repeatable per-flow playbook, with every mailbox permission **scoped to least privilege** (per #4), **parity-validated** so nothing breaks silently, and **governed** so the estate does not re-accumulate the risk.

Honest scope

Power Automate flows do not always take an Azure managed identity the way Logic Apps do — some need a re-platform, not a swap; the program flags these in classification. Connector deprecation timelines, exact Graph permission names, change-notification lifetimes, the Application-Access-Policy-vs-Graph-RBAC choice, and national-cloud specifics are **verified against current Microsoft documentation** at migration time, never asserted. The per-flow engineering depth lives in the sibling products; this program orchestrates them.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with a pilot-first gate and a documented per-flow rollback.

Outcome

An Office 365 connector estate re-platformed onto managed-identity Microsoft Graph — credential-free, least-privilege, parity-proven, and governed — deployable by a single administrator and validated on a pilot before it scales.

1. Solution Overview — Modernizing Office 365 Connector Workflows to Microsoft Graph

Why this is a program, not a one-off

One Logic App hitting the Office 365 connector's authentication wall is a **fix**. Dozens of Logic Apps and Power Automate flows — all built on the Office 365 Outlook / Office 365 Users / Excel Online connectors, all authenticated as a user or a shared service account — is a **program**. Organizations accumulate these connector-based flows over years: a mail-sending flow here, a mailbox-polling flow there, a "when a new email arrives" trigger somewhere else. Each one holds a standing credential (a signed-in user's token or a service account's password), each one is a piece of the estate nobody re-authorized, and together they are a large, quiet authentication-risk surface that no single fix retires.

This solution is the **migration program** that retires that surface at estate scale. It does three things a single-flow fix cannot: it **inventories** every connector-based flow and how each one authenticates; it **classifies** them by risk and by how cleanly each maps onto a managed-identity Microsoft Graph design; and it **re-platforms** them in **waves** — pilot first — onto managed-identity Graph workflows, validating parity so nothing silently breaks. The program-level value is the **inventory + classification + wave rollout + governance** discipline; the per-flow engineering depth lives in the sibling products this program orchestrates.

The inventory → classify → migrate → govern arc

The program follows one arc from a sprawling connector estate to a governed, credential-free end state:

1. **Inventory** — enumerate every Logic App and, where the platform allows, every Power Automate flow that uses an Office 365 connector, and record **how each authenticates** (interactive user, shared service account, or stored secret). The output is the **Connector Flow Inventory – Estate – Program** register.

2. **Classify** — score each flow by **risk** (standing credentials, over-scoped access, a shared service account behind many flows) and by **Graph-migratability** (does the flow's behaviour map cleanly onto a managed-identity Graph pattern, or does it need redesign). The output is the **Migration Register – Estate – Program**.

3. **Migrate** — run the **per-flow playbook**: for each flow, pick the matching sibling pattern, scope its Graph permission to only what it needs, cut it over, and validate parity. This is where the program **references, and does not reproduce**, the per-flow products (below).

4. **Govern** — after the estate is migrated, keep it that way: prevent regression to connector-based auth, and report any flow that still carries a connector or a stored credential.

The repeatable delivery shape across the whole program is **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a **pilot-first gate** and a **per-flow rollback**.

The per-flow playbook leans on the sibling products (reference, don't re-teach)

The program does not re-teach how to build a single managed-identity Graph workflow — that depth is already productized. For each flow, the playbook says **"apply the pattern from the relevant product below,"** then focuses on the program-level concerns (sequencing, parity, rollback, governance):

- **Sending / reading / moving mail, or general mailbox automation** → apply **"Exchange Online Mailbox Automation with Logic Apps Managed Identity — Replacing the O365 Connector with Microsoft Graph"** (sibling #1).
- **Event-driven "when a new email arrives" triggers** → apply **"Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity)"** (sibling #2).
- **Scheduled / recurring mailbox polling** → apply **"Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)"** (sibling #3).
- **The managed-identity + app-permission foundation any flow needs** → apply **"Managed Identities for Azure Logic Apps & Functions — Authenticating to Microsoft 365 Without Stored Credentials"** (sibling #5).
- **Scoping the Graph application permission down to a single mailbox (or a defined set)** → **defer to "Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online"** (sibling #4). The program never grants a tenant-wide mailbox permission where a scoped one will do; it references #4 for exactly how to constrain it.

The through-line the program owns is the **discipline**: which flow migrates in which wave, how parity is proven, and how the end state is governed. The per-flow "how" is in #1/#2/#3/#4/#5.

Logic Apps and Power Automate are not the same — treat the difference honestly

Azure **Logic Apps** and **Power Automate** flows both use Office 365 connectors, but they **do not authenticate the same way**, and this shapes the migration:

- **Azure Logic Apps** can be assigned an **Azure managed identity** (system- or user-assigned) and can call Microsoft Graph with an application permission — this is the clean managed-identity Graph end state the sibling products build.
- **Power Automate** flows typically authenticate through **connection references** tied to a user or a service account, and **do not consume Azure managed identities** the same way Logic Apps do. A Power Automate flow's "modernized" target is therefore not always a drop-in managed-identity swap; it may need to be **re-platformed into a Logic App**, re-pointed at a **least-privilege service principal / connection**, or redesigned.

Because the exact capabilities drift between platform versions and connector releases, **the specific answer for any given flow — whether a particular Office 365 connector trigger/action supports managed identity, and what the correct modern target is for a Power Automate flow — must be verified against current Microsoft documentation** at migration time. What is stable, and what this program commits to, is the **target end state** (a Graph-based workflow with **no stored user or service-account credential**) and the **migration discipline** (inventory → classify → pilot → validate → wave → govern).

Benefits

• **The whole estate is seen, not guessed** — a single register of every connector-based flow and its authentication model, instead of tribal knowledge.

• **Risk is ranked, not uniform** — the flows with standing service-account credentials and the broadest access are migrated first.

• **Credentials are eliminated at scale** — the end state is managed-identity Graph (Logic Apps) and least-privilege service principals (where Power Automate can't take a managed identity), with **no stored passwords or user tokens** behind automation.

• **Least privilege by construction** — every migrated flow's Graph **application** permission is scoped (per sibling #4) to only the mailbox(es) it needs, not the whole tenant.

• **Nothing breaks silently** — each flow is migrated with a parity check and a per-flow rollback; the pilot proves the method before the estate follows.

• **It stays fixed** — governance reporting catches any new or reverted connector-based flow before it becomes the next quiet risk.

Limitations (be honest with buyers)

• **Power Automate flows may not be a like-for-like managed-identity swap.** Where a flow cannot take an Azure managed identity, the modern target is a re-platform or a least-privilege service-principal connection — more work than a Logic Apps swap. The program flags these in classification rather than pretending they are identical.

• **Enumerating Power Automate flows is not the same as enumerating Logic Apps.** Logic Apps are Azure resources the discovery script can list directly; Power Automate flows live in the Power Platform and require a Power Platform admin enumeration method. The exact enumeration surface is a **verify-against-current-documentation** item.

• **The per-flow depth is in the sibling products.** This program orchestrates and sequences; the detailed build of any single managed-identity Graph workflow is delivered by #1/#2/#3/#5 (and scoped by #4). It references them; it does not reproduce them.

• **Graph does not always expose an identical shape to the old connector action** — some flows need a small redesign to reach behavioural parity. The classification step surfaces these before the wave, not during it.

• **This is a migration program, not a net-new automation build** — it re-platforms existing behaviour; it does not invent new automation.

• **Connector deprecation timelines, Graph permission names, change-notification lifetimes, and national-cloud endpoint specifics are not asserted here** — they are explicitly deferred to current Microsoft documentation, because they change.

The standard deployment model

> ***Inventory the connector estate and its auth → classify by risk and Graph-migratability → configure the pilot cohort's managed-identity Graph targets (per the sibling patterns, scoped per #4) → pilot the migration on a small cohort → validate parity and confirm no residual credentials → expand in waves with per-flow rollback → govern the end state to prevent regression.*** *This is the repeatable pattern* ***Assess → Configure → Pilot → Validate → Expand****, with a* ***pilot-first gate*** *and a* ***documented per-flow rollback****.*

2. Requirements Guide — Modernizing Office 365 Connector Workflows to Microsoft Graph

Verify **every** requirement before running the program. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md` (G1–G6). Objects created during the program are named `<Purpose> – <Scope> – <Stage>` and reused verbatim.

> ***Verify-against-Microsoft flags appear throughout.*** *Whether a specific Office 365 connector trigger/action supports managed identity, the exact Microsoft Graph application permission names/scopes, and the Power Platform enumeration surface all drift between platform and connector versions —* ***confirm each against current Microsoft documentation at migration time****; this guide states the target and flags the specifics.*

2.1 Read access to enumerate the Logic Apps estate

Description: An account with **Reader** (or higher) across the Azure subscriptions/resource groups that hold Logic Apps, so the program can list every Logic App and inspect its workflow definition and connections. **Why:** The inventory step must find every Logic App that uses an Office 365 connector and record how it authenticates; that requires read access to the workflow definitions and their API connections across the estate. **Verify:** `portal.azure.com` → **Subscriptions** → **Access control (IAM)** shows **Reader** at the needed scopes; run `O365GRAPH-Discovery.ps1` (read-only) — it lists Logic Apps and their Office 365 connections. **Implement-if-missing:** Assign **Reader** at the narrowest scope that still covers every subscription/resource group holding Logic Apps; prefer PIM-eligible activation over standing access.

2.2 Read access to enumerate Power Automate flows (Power Platform)

Description: A **Power Platform administrator** (or equivalent read role) able to enumerate flows and their connection references across the environments in scope. **Why:** Power Automate flows are **not** Azure resources — they live in the Power Platform and are enumerated through a Power Platform admin method, separate from the Azure Logic Apps inventory. Without this role the inventory misses half the estate. **Verify:** `admin.powerplatform.microsoft.com` resolves and lists the in-scope environments; confirm the account can view flows and connections. **Implement-if-missing:** Grant the Power Platform admin (or

a scoped read) role for the in-scope environments. **The exact enumeration method (admin center export, Power Platform admin PowerShell/CLI, or Dataverse query) is a** `verify against current Microsoft documentation` **item** — the capability and command names change between platform versions.

2.3 Rights to create managed identities and grant application permissions

Description: Rights to create/assign **Azure managed identities** for Logic Apps (Contributor on the Logic App), and an identity role able to **grant Microsoft Graph application permissions and admin-consent** them (e.g. Privileged Role Administrator / Global Administrator for consent, or a delegated app-management role). **Why:** The modern end state assigns each migrated Logic App a managed identity and grants it a **scoped Graph application permission**; creating the identity and consenting the permission both require these rights. **Verify:** Confirm the operating account can add a managed identity to a Logic App and grant/admin-consent an application permission (Entra → **Enterprise applications** / **App registrations**); `O365GRAPH-Set-AppRole.ps1 -WhatIf` reports whether the intended app-role assignment is permitted. **Implement-if-missing:** Assign the minimum roles at the narrowest scope; separate "create the identity" (Contributor on the Logic App) from "consent the permission" (a consent-capable role) where your org splits those duties. **The exact Graph application permission required for a given flow (e.g. a mail permission) is a** `verify against current Microsoft documentation` **item** — confirm the current least-privilege permission name and use it.

2.4 Rights to scope mailbox access (defer the mechanism to sibling #4)

Description: The ability to constrain a migrated flow's Graph **application** mailbox access to only the mailbox(es) it needs — via an **Application Access Policy** or the newer Graph **RBAC-for-Exchange** model. **Why:** A Graph mail application permission is tenant-wide by default; least privilege requires scoping it down. This program **defers the "how" to sibling #4** ("Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online") and treats the scoping as a required gate on every mail-touching flow. **Verify:** Confirm Exchange Online admin rights sufficient to create the scoping construct; sibling #4 details the verification. **Implement-if-missing:** Follow sibling #4. **Which mechanism to use — the established Application Access Policy vs the newer Graph RBAC-for-Exchange model — is a** `verify against current Microsoft documentation` **item**; pick the current recommended one for your tenant and cloud.

2.5 Microsoft.Graph PowerShell and an enumeration method for Power Automate

Description: The **Microsoft.Graph** PowerShell SDK installed on the operating workstation, plus **Az** PowerShell (for Logic Apps enumeration) and whatever Power Platform enumeration method your tenant supports. **Why:** The program's scripts (`O365GRAPH-*.ps1`) use Microsoft.Graph for identity/permission checks and Az for Logic Apps discovery; Power Automate enumeration uses a separate Power Platform method. **Verify:** `Get-Module Microsoft.Graph -ListAvailable` and `Get-Module Az -ListAvailable` return versions; run `O365GRAPH-Discovery.ps1` — it reports which modules/methods are available. **Implement-if-missing:** `Install-Module Microsoft.Graph -Scope CurrentUser` and `Install-Module Az -Scope CurrentUser`. **The Power Platform enumeration module/command is a** `verify against current Microsoft documentation` **item** — install the current supported one.

2.6 A named pilot cohort

Description: A small, representative set of connector-based flows chosen as the first migration wave — the **PILOT – Connector Migration – Pilot** cohort — covering the flow types that matter (a send flow, an event/trigger flow, a polling flow) and, ideally, both a Logic App and a Power Automate flow. **Why:** The program is pilot-first: the method is proven on a small cohort and gated before any wave. A representative pilot exercises every per-flow pattern once. **Verify:** A written pilot list exists; each pilot flow is tagged in the **Migration Register – Estate – Program**. **Implement-if-missing:** Pick 3–8 flows spanning the sibling patterns (#1 send/read, #2 change-notification trigger, #3 polling) and both platforms; record them as the pilot cohort.

2.7 Change control, parity baseline, and per-flow rollback

Description: An approved change window per wave, a captured **parity baseline** for each flow (what the legacy connector flow does — its trigger, its outputs, its schedule), and a documented **per-flow rollback** (re-enable the legacy flow, disable the migrated one) before any cutover. **Why:** A migrated flow must match the legacy flow's behaviour; parity can only be judged against a captured baseline, and a cutover that breaks parity must be reversible per flow. **Verify:** An approved change record, a per-flow parity baseline, and a written rollback exist (per `global-requirements-azure-solutions.md` G5/G6). **Implement-if-missing:** Capture each pilot/wave flow's behaviour with `O365GRAPH-Discovery.ps1` output as the baseline; write the per-flow rollback (the legacy flow is **disabled, not deleted**, until its migrated replacement passes validation and a grace period elapses).

2.8 National-cloud endpoint confirmation (if applicable)

Description: For government/sovereign tenants (GCC / GCC High / DoD), confirmation of the correct Graph, Azure, and Power Platform endpoints and any feature-parity gaps. **Why:** Government clouds use distinct endpoints and can lag commercial on connector/Graph features; a step correct for commercial can be wrong for a sovereign cloud. **Verify:** Confirm the tenant's cloud and the matching endpoint set (per `global-requirements-azure-solutions.md` G4). **Implement-if-missing:** Target the correct sovereign endpoints. **National-cloud endpoints and feature parity are a** `verify against current Microsoft documentation` **item** — do not assume commercial specifics apply.

Checklist

- Reader across the Logic Apps estate — `O365GRAPH-Discovery.ps1`
- Power Platform admin / read to enumerate Power Automate flows (method: verify)
- Rights to create managed identities + grant/consent Graph application permissions
- Rights to scope mailbox access (mechanism deferred to sibling #4; verify AAP vs Graph RBAC)
- Microsoft.Graph + Az PowerShell installed; Power Platform enumeration method (verify)
- Named pilot cohort `PILOT – Connector Migration – Pilot`
- Change window, per-flow parity baseline, and per-flow rollback documented
- National-cloud endpoints confirmed if government (verify)

Global Requirements — Azure Solutions

These requirements are common to **every** Azure Solutions solution in the standard program model. They are the baseline platform prerequisites that any Azure/Microsoft 365 engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named `<Purpose> – <Scope> – <Stage>` and reused verbatim throughout the engagement.*

G1. A Microsoft 365 / Entra tenant

• **Description:** An active Microsoft 365 / Microsoft Entra tenant for the cloud(s) in scope (commercial and/or government).

• **Why:** All identity, licensing, and service configuration is anchored to a tenant; nothing can be configured without one.

• **Verify:** Sign in to the relevant admin center (`https://admin.microsoft.com`) and confirm the tenant resolves and is active under **Settings** → **Org settings**.

• **Implement-if-missing:** Provision the tenant through the appropriate channel (commercial: standard; government: via an authorized AOS-G reseller).

G2. Administrative roles (least privilege)

• **Description:** Appropriately scoped admin roles for the work — Global Administrator for tenant-wide changes, and service-specific roles (Exchange, SharePoint, Teams, Azure Owner/Contributor) for day-to-day.

• **Why:** Every configuration and migration action requires a role with sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** In Entra → **Roles & admins**, confirm the operating accounts hold the required roles. In Azure → **Subscriptions** → **Access control (IAM)**, confirm RBAC assignments.

• **Implement-if-missing:** Assign the minimum roles needed; prefer eligible (PIM) activation over standing access where available.

G3. Licensing baseline

• **Description:** Sufficient Microsoft 365 and/or Azure licensing/credit for all in-scope users and workloads.

• **Why:** Features and capacity (mailboxes, SharePoint storage, AVD, VMs) are gated by license SKUs and subscription quota.

• **Verify: Billing** → **Licenses** for M365; **Subscriptions** → **Usage + quotas** for Azure. Compare assigned/available against the in-scope object count.

• **Implement-if-missing:** Procure the correct SKUs and request quota increases sized to the workload (add headroom for shared/resource objects).

G4. Network connectivity to Microsoft 365 / Azure endpoints

• **Description:** Reliable network egress to the required Microsoft 365 and Azure service endpoints (and the sovereign `.us` endpoints for government clouds), with sufficient bandwidth.

• **Why:** All cloud administration and any data movement depends on connectivity to the correct endpoint set; government clouds use distinct endpoints from commercial.

• **Verify:** Confirm reachability to the relevant endpoints (commercial `*.microsoft.com` / government `*.microsoft.us`, `*.azure.us`, `*.office365.us`). Measure sustained bandwidth against the data volume.

• **Implement-if-missing:** Open firewall/proxy allowances for the correct endpoint set; scale bandwidth or add ExpressRoute (matching the cloud — Gov ExpressRoute for Gov regions).

G5. Change control & maintenance window

• **Description:** An approved change-management process with scheduled maintenance windows and a stakeholder communications plan.

• **Why:** Cloud changes can be outward-facing and disruptive; controlled windows and comms reduce business impact and satisfy governance/compliance.

• **Verify:** Confirm an approved change record and a scheduled window for each disruptive action.

• **Implement-if-missing:** Stand up a lightweight change/approval process; schedule windows; prepare user comms templates.

G6. Backup / rollback posture

• **Description:** A defined backup and rollback position for any change — config export, data backup, and a documented back-out procedure before any disruptive action.

• **Why:** Cloud changes are not always reversible by default; a tested rollback is the safety net for cutovers and bulk operations.

• **Verify:** Confirm pre-change config/state is captured and a written rollback exists for each disruptive change.

• **Implement-if-missing:** Export current configuration/state; document and (where feasible) rehearse the rollback before executing the change.

3. Implementation Guide — Modernizing Office 365 Connector Workflows to Microsoft Graph

This is the **program runbook**. Follow the phases **in order**. The program follows the standard model — **Inventory** → **Classify** → **Configure (pilot targets)** → **Pilot** → **Validate (pilot gate)** → **Expand (waves)** → **Govern** — mapping to **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, test, and rollback.

> ***Sibling boundary (read first):*** *the* ***program-level*** *work — inventory, classification, wave sequencing, parity, rollback, governance — is given here as exact steps. The* ***per-flow engineering*** *(how to build*

*one managed-identity Graph workflow) is **not reproduced**; each flow's cutover step says *"apply the pattern from the relevant sibling product"* and *"scope the permission per sibling #4."* See* `01-solution-overview.md` *for the sibling map.*

***Verify-against-Microsoft (read first):** connector managed-identity support, exact Graph permission names, change-notification lifetimes, the Application-Access-Policy-vs-Graph-RBAC choice, Logic Apps Standard-vs-Consumption differences, the Power Platform enumeration method, and national-cloud endpoints are **confirmed against current Microsoft documentation at migration time** — this runbook states the target and flags the specifics.*

Naming (set once, reuse verbatim)

• Inventory register (CSV): `Connector-Flow-Inventory – Estate – Program.csv` — the **Connector Flow Inventory – Estate – Program**.

• Classified migration register (CSV): `Migration-Register – Estate – Program.csv` — the **Migration Register – Estate – Program**.

• Pilot cohort (Entra security group for pilot owners/testers, and the tag in the register): `PILOT – Connector Migration – Pilot`.

• Wave cohorts (Entra security group + register tag): `Wave NN – Connector Migration – Wave NN` (`Wave 01`, `Wave 02`, …).

• Per migrated Logic App's managed identity: `<FlowName> – Graph MI – Target` (one per migrated Logic App; reuse the exact Logic App name).

• The scoped mailbox-access construct for a migrated flow: named and built **per sibling #4** (Application Access Policy or Graph RBAC scope), tagged in the register against the flow.

Phase 1 — Inventory the connector estate and its authentication

Goal: one authoritative register of every connector-based flow and **how each authenticates**.

1. **Logic Apps side (Azure):** run the read-only `scripts\O365GRAPH-Discovery.ps1`. It enumerates Logic Apps across the in-scope subscriptions, flags those whose workflow definition uses an **Office 365 connector** (Office 365 Outlook / Office 365 Users / Excel Online), and records **how each authenticates** — the API connection's identity (interactive user, shared service account, or stored secret) vs. an existing managed identity.

2. **Power Automate side (Power Platform):** enumerate flows and their **connection references** across the in-scope environments using your Power Platform admin method, and record the same fields (which connector, authenticated as whom). **The exact enumeration method is a** `verify against current Microsoft documentation` **item** — the discovery script flags where the Power Platform data must be supplied.

3. **Merge** both sides into `Connector-Flow-Inventory – Estate – Program.csv`, one row per flow, with at minimum: flow name, platform (Logic App / Power Automate), connector(s) used, auth model (user / service account / secret), owner, and the mailbox(es) or resources it touches.

4. **Reconcile** service accounts: flag every flow that shares a service-account credential — one compromised account behind many flows is a top-risk finding.

Exit gate: a signed-off `Connector-Flow-Inventory – Estate – Program.csv` covering 100% of in-scope Logic Apps and Power Automate flows, with an auth model recorded for each.

Phase 2 — Classify by risk and Graph-migratability

Goal: turn the inventory into a risk-ranked, migration-sequenced plan.

1. Run `scripts\O365GRAPH-Classify.ps1 -InputCsv "Connector-Flow-Inventory – Estate – Program.csv" -OutputCsv "Migration-Register – Estate – Program.csv"`. It scores each flow on two axes:

• **Risk** — standing credentials (service account > interactive user > already-managed-identity), over-scope (touches many mailboxes / broad access), and shared-service-account blast radius.

• **Graph-migratability** — how cleanly the flow maps to a managed-identity Graph pattern: **Clean** (a Logic App whose action has a direct Graph equivalent and can take a managed identity), **Redesign** (behaviour maps to Graph but the flow needs restructuring), or **Re-platform** (a Power Automate flow that cannot take an Azure managed identity — see the note below).

1. **Assign each flow its sibling pattern** in the register — the migration target pattern:

• send/read/move mail or general mailbox automation → **#1**;

• "when a new email arrives" / event-driven → **#2**;

• scheduled/recurring polling → **#3**;

• the managed-identity + app-permission foundation underneath → **#5**;

• and every mail-touching flow is tagged **"scope per #4."**

1. **Power Automate honesty:** flows that **cannot consume an Azure managed identity** are tagged **Re-platform** — their modern target is a re-platform into a Logic App, or a least-privilege service-principal/connection, not a drop-in managed-identity swap. **Whether a given Power Automate flow or Office 365 connector trigger can take a managed identity is a** `verify against current Microsoft documentation` **item** — confirm before committing the target.

2. **Sequence:** sort the register — highest risk + Clean migratability first (fast, high-value wins), Redesign/Re-platform into later waves. Tag the pilot cohort `PILOT – Connector Migration – Pilot` from the top of the list, spanning the sibling patterns.

Exit gate: an approved `Migration-Register – Estate – Program.csv` with a risk score, a migratability class, an assigned sibling pattern, and a wave tag per flow.

Phase 3 — Configure the pilot cohort's managed-identity Graph targets

Goal: build the modern target for each pilot flow, using the sibling patterns, before any cutover. **Prerequisites 2.1–2.6 must be satisfied.**

For **each** flow in `PILOT – Connector Migration – Pilot`:

1. **Capture the parity baseline** (requirement 2.7): record the legacy flow's trigger, schedule, inputs, and outputs from `O365GRAPH-Discovery.ps1`, so parity can be judged later.

2. **Build the managed-identity Graph target by applying the assigned sibling pattern** — do **not** re-derive it here:

• Stand up the managed-identity + application-permission foundation per **sibling #5**, naming the Logic App's identity `<FlowName> – Graph MI – Target`.

• Build the flow's Graph logic per **sibling #1 / #2 / #3** (send/read, change-notification, or polling) as the flow's type dictates.

1. **Scope the mailbox permission per sibling #4** — constrain the flow's Graph mail **application** permission to only the mailbox(es) the flow touches (Application Access Policy or Graph RBAC-for-Exchange). **Which mechanism is a** `verify against current Microsoft documentation` **item.** Never grant tenant-wide mailbox access where a scoped grant suffices.

2. **Assign the application permission to the managed identity** with `scripts\O365GRAPH-Set-AppRole.ps1` (defaults to `-WhatIf` — review the plan, then re-run with `-Apply` once approved). **The exact permission name is a** `verify against current Microsoft documentation` **item** — supply the current least-privilege permission.

3. **Leave the legacy flow enabled** — the migrated target is built alongside it; cutover happens in the pilot phase.

Exit gate: each pilot flow has a built managed-identity Graph target, a scoped permission (per #4), and a captured parity baseline — legacy flows still running.

Phase 4 — Pilot the migration

Goal: cut over the pilot cohort and prove the method.

For each flow in `PILOT – Connector Migration – Pilot`, run the **per-flow playbook**:

1. **Enable** the migrated managed-identity Graph target.

2. **Cut over:** disable the legacy connector flow (do **not** delete it) so only the migrated flow runs. For an event/trigger flow, follow **sibling #2** for the change-notification handover; for a polling flow, **sibling #3**.

3. **Validate parity immediately** with `05-testing.md` Test 2 and `scripts\O365GRAPH-Validate.ps1` for that flow — the migrated flow must produce the same behaviour as the captured baseline.

4. **Rollback if parity breaks:** re-enable the legacy flow, disable the migrated one (requirement 2.7). Fix, then retry. The legacy flow is only retired after the migrated flow passes and a grace period elapses.

Capture real timings and any redesign surprises to refine the wave plan. **No production wave starts until Phase 5 passes.**

Phase 5 — Validate the pilot (the gate)

Goal: the program's go/no-go.

1. Run `scripts\O365GRAPH-Validate.ps1` across the pilot cohort — every migrated flow must be **PASS**: it uses a managed identity + Graph, has **no stored-credential connection**, and its scoped

permission is present.

2. Run `scripts\O365GRAPH-Report.ps1` — confirm the pilot flows show as migrated with no residual connector/credential.

3. Complete the manual parity gates in `05-testing.md`. **The pilot gate passes only when every pilot flow passes parity and shows no residual credential.** Record results as `Validation Results – Program – Pilot`.

Do not expand until this gate is green.

Phase 6 — Expand in waves

Goal: migrate the rest of the estate, wave by wave, with per-flow rollback.

For each wave `Wave NN – Connector Migration – Wave NN` (in the sequence set in Phase 2):

1. **Configure** each wave flow's managed-identity Graph target exactly as Phase 3 (apply the assigned sibling pattern; scope per #4; assign the permission with `O365GRAPH-Set-AppRole.ps1`).

2. **Cut over** each flow exactly as Phase 4's per-flow playbook (enable migrated, disable legacy, validate parity, rollback per flow if it breaks).

3. **Validate the wave** with `O365GRAPH-Validate.ps1` and the `05-testing.md` gates — every flow PASS, no residual credential.

4. **Track progress** with `O365GRAPH-Report.ps1` (total connector flows, migrated, remaining, risk buckets).

5. **Retire legacy flows** only after their migrated replacements pass and the grace period elapses — disable first, delete last, under change control.

Exit gate per wave: every flow in the wave validated and parity-confirmed; report shows the wave migrated with no residual connector/credential.

Phase 7 — Govern the end state (prevent regression)

Goal: keep the estate credential-free after migration.

1. **Baseline the end state:** run `O365GRAPH-Report.ps1` at fleet completion — the target is **zero** connector-based flows with stored credentials remaining (or an approved, tracked exception list).

2. **Scheduled governance scan:** run `O365GRAPH-Discovery.ps1` + `O365GRAPH-Validate.ps1` on a recurring cadence to catch any **new** connector-based flow or any migrated flow that **reverted** to a stored-credential connection.

3. **Standard for new automation:** adopt the rule that new mailbox/M365 automation is built managed-identity-Graph-first (per the sibling patterns), scoped per #4 — so the estate does not re-accumulate the risk this program retired.

4. **Report the trend** each cycle: a falling count of connector/credential flows is the evidence the program worked and the governance is holding.

Order of operations

`Inventory (Logic Apps + Power Automate) → classify (risk + migratability + sibling pattern) → configure pilot targets (apply #1/#2/#3/#5, scope per #4) → pilot cutover + parity → pilot gate → expand in waves (per-flow rollback) → govern (prevent regression)`

5. Testing Guide — Modernizing Office 365 Connector Workflows to Microsoft Graph

Run after the pilot cutover (Implementation Phase 5) and again after each wave (Phase 6) and as the governance scan (Phase 7). Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names. Every test is **PASS/FAIL**.

Test 1 — Inventory completeness

Do: Compare `Connector-Flow-Inventory – Estate – Program.csv` against a fresh `scripts\O365GRAPH-Discovery.ps1` run (Logic Apps) plus the Power Platform enumeration (Power Automate). **Expected (PASS):** Every in-scope Logic App and Power Automate flow that uses an Office 365 connector appears in the register, each with an auth model recorded. **FAIL** if any connector-based flow is missing or has no auth model.

Test 2 — Per-flow parity (migrated matches legacy)

Do: For the migrated flow, compare its behaviour against the captured parity baseline (requirement 2.7) — same trigger condition, same outputs, same schedule/cadence. Exercise it end-to-end (send a test mail / raise the trigger event / let the poll run). **Expected (PASS):** The migrated flow produces the **same** observable result as the legacy connector flow did. **FAIL** if any output, trigger behaviour, or schedule differs.

Test 3 — Migrated flow uses managed identity + Graph

Do: Run `scripts\O365GRAPH-Validate.ps1` for the flow; or inspect the Logic App's workflow definition and identity. **Expected (PASS):** The migrated Logic App calls **Microsoft Graph** authenticated by its **managed identity** `<FlowName> – Graph MI – Target`. **FAIL** if it still calls an Office 365 connector or authenticates any other way.

Test 4 — No residual stored credentials in the migrated cohort

Do: Run `scripts\O365GRAPH-Validate.ps1` across the migrated cohort; confirm no API connection is backed by a user token or service-account secret. **Expected (PASS): Zero** migrated flows carry a stored-credential connection. **FAIL** if any migrated flow still has a user/service-account connection.

Test 5 — Scoped mailbox permission (least privilege, per #4)

Do: Confirm the flow's Graph mail **application** permission is scoped to only the mailbox(es) it needs (Application Access Policy or Graph RBAC-for-Exchange, per sibling #4). Attempt access to an

out-of-scope mailbox with the flow's identity. **Expected (PASS):** The identity can access only its scoped mailbox(es); an out-of-scope mailbox is **denied**. **FAIL** if the permission is tenant-wide or reaches an out-of-scope mailbox. *(Mechanism specifics: verify against current Microsoft documentation.)*

Test 6 — Per-flow rollback works

Do: On a test flow, exercise the rollback (re-enable the legacy connector flow, disable the migrated one) and confirm the legacy flow resumes correct behaviour. **Expected (PASS):** The legacy flow was **disabled, not deleted**, and resumes on rollback; the migrated flow can be re-cut-over afterward. **FAIL** if rollback is not possible or the legacy flow was already deleted.

Test 7 — Program progress / no regression

Do: Run `scripts\O365GRAPH-Report.ps1`. **Expected (PASS):** The report shows total connector flows, migrated, and remaining, with risk buckets; the governance scan shows **no new** connector-based flow and **no migrated flow reverted** to a stored credential. **FAIL** if a new connector/credential flow appeared or a migrated flow regressed.

Pass/fail gate

Gate	Requirement
Pilot gate (go/no-go)	Tests 1–6 PASS for every flow in `PILOT – Connector Migration – Pilot`
Wave gate	Tests 2–5 PASS for every flow in the wave; Test 6 spot-checked
Governance gate	Test 7 PASS each cycle — falling connector/credential count, no regression

6. Troubleshooting Guide — Modernizing Office 365 Connector Workflows to Microsoft Graph

Use `O365GRAPH-Discovery.ps1` / `O365GRAPH-Classify.ps1` for inventory/classification state, `O365GRAPH-Validate.ps1` for PASS/FAIL per flow, `O365GRAPH-Report.ps1` for program progress, and `O365GRAPH-Set-AppRole.ps1` (`-WhatIf` first) for the app-role grant. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names. Volatile Microsoft specifics are flagged **verify against current documentation**.

A. A flow was missed in the inventory

Symptom	Cause	Remediation
A connector-based flow surfaces after migration "completed"	Discovery covered only the Logic Apps (Azure) side; the Power Automate side wasn't enumerated	Re-run `O365GRAPH-Discovery.ps1` **and** the Power Platform enumeration for every in-scope environment; merge both into `Connector-Flow-Inventory – Estate – Program.csv`. The Power Platform method is a **verify-against-docs** item — confirm it lists all environments
A Logic App uses O365 but wasn't flagged	The connector id wasn't in the match list, or the connection is referenced indirectly	Confirm the connector identifiers in `O365GRAPH-Discovery.ps1` against current Microsoft connector names (verify); inspect the workflow's connections blade directly
A subscription/resource group is absent	Reader wasn't granted at that scope	Grant Reader at the missing scope (requirement 2.1) and re-run discovery — inventory completeness (Testing Test 1) must pass before classifying

B. Parity break after cutover

Symptom	Cause	Remediation
Migrated flow runs but produces a different result than the legacy flow	No captured parity baseline, or a Graph shape that doesn't match the old connector action 1:1	**Roll back per flow** (re-enable the legacy flow, disable the migrated one — requirement 2.7), compare against the captured baseline, adjust the Graph logic per the assigned sibling pattern (#1/#2/#3), then re-cut-over and re-run Testing Test 2
Event-driven flow misses events after cutover	Change-notification subscription not created/renewed correctly	Follow **sibling #2** for the change-notification handover and renewal; the max subscription lifetime + renewal cadence are **verify-against-docs** items
Polling flow drifts on schedule/dedupe	Polling cadence or state handling differs from the legacy flow	Follow **sibling #3** for the polling pattern; confirm the schedule matches the baseline

Symptom	Cause	Remediation
You can't roll back	The legacy flow was deleted, not disabled	Legacy flows must be **disabled, not deleted**, through the grace period (Best Practices). Rebuild from source/export if deleted; re-establish the rollback before the next cutover

C. Power Automate flow can't use a managed identity

• **This is expected for many Power Automate flows.** Power Automate typically authenticates through **connection references** tied to a user or service account and does **not** consume Azure managed identities the way Logic Apps do — **whether a given flow/connector can is a verify-against-current-documentation item.**

• **Route to the alternative** the classification tagged **Re-platform**: either (a) **re-platform** the flow into an **Azure Logic App** and apply the managed-identity Graph pattern (siblings #1/#2/#3/#5), or (b) re-point it at a **least-privilege service principal / connection** so it no longer runs on a user or shared service-account credential. Pick the option that fits the flow's owner and platform; confirm the current capability against Microsoft docs before committing.

• Do **not** record a Re-platform flow as "migrated" until it meets the same end-state test as a Logic App: no stored user/service-account credential (Testing Test 4).

D. Permission / consent gaps

Symptom	Cause	Remediation
`O365GRAPH-Set-AppRole.ps1` throws "app-role not found as Application permission"	Wrong or renamed Graph permission name	Confirm the current least-privilege application permission name against Microsoft documentation (verify) and pass it as `-GraphAppRole`
Migrated flow gets 403 from Graph	The app-role was never consented, or is scoped away	Confirm the app-role assignment exists (`O365GRAPH-Set-AppRole.ps1 -WhatIf` reports it); ensure a consent-capable role admin-consented it (requirement 2.3)
Flow can read a mailbox it shouldn't	The Graph mail permission is tenant-wide and not scoped	Apply the scoping construct **per sibling #4** (Application Access Policy or Graph RBAC-for-Exchange) to constrain the identity to only its mailbox(es); re-run Testing Test 5. Which mechanism to use is a **verify-against-docs** item

Symptom	Cause	Remediation
Can't create the managed identity or grant consent	Missing Contributor on the Logic App or a consent-capable role	Assign the minimum roles at the narrowest scope (requirement 2.3); separate "create identity" from "consent permission" where duties are split

E. Regression to connector-based auth

- **Symptom:** the governance scan (`O365GRAPH-Report.ps1` / `O365GRAPH-Validate.ps1`) shows a **new** connector-based flow, or a migrated flow that **reverted** to a stored-credential connection.
- **Cause:** new automation built the old way, or a migrated flow edited back to an Office 365 connection.
- **Remediation:** treat it as a wave-of-one — inventory it, classify it, migrate it with the per-flow playbook, and scope it per #4. Reinforce the standard that new M365 automation is built managed-identity-Graph-first (Implementation Phase 7). A rising migrated count that later falls is the signal governance caught a regression.

F. Diagnostics quick reference

```
# Re-baseline the Logic Apps side of the inventory
.\O365GRAPH-Discovery.ps1 -OutputCsv '.\Connector-Flow-Inventory - Estate - Program.csv'

# Re-score risk + migratability
.\O365GRAPH-Classify.ps1 -InputCsv '.\Connector-Flow-Inventory - Estate - Program.csv' `
                         -OutputCsv '.\Migration-Register - Estate - Program.csv'

# PASS/FAIL a migrated flow (managed identity + Graph, no stored credential)
.\O365GRAPH-Validate.ps1 -LogicAppName 'Contoso-MailNotify' -ResourceGroup 'rg-automation'

# Program dashboard / governance scan
.\O365GRAPH-Report.ps1 -Register '.\Migration-Register - Estate - Program.csv'

# Grant the app-role (dry run first, then -Apply); scope the mailbox reach per sibling #4
.\O365GRAPH-Set-AppRole.ps1 -ManagedIdentityObjectId '<mi sp object id>' -GraphAppRole 'Mail.Read'
.\O365GRAPH-Set-AppRole.ps1 -ManagedIdentityObjectId '<mi sp object id>' -GraphAppRole 'Mail.Read' -Apply
```

G. When to escalate

If the inventory is complete, the managed identity is assigned, the app-role is consented and scoped per #4, and a migrated flow still fails parity or Graph calls still 403 after the normal propagation window, collect `O365GRAPH-Validate.ps1` and `O365GRAPH-Report.ps1` output plus the flow's definition and open a Microsoft support case. For Power Automate managed-identity capability questions and national-cloud endpoint parity, confirm against current Microsoft documentation before escalating.

7. Best Practices — Modernizing Office 365 Connector Workflows to Microsoft Graph

- **Inventory before you migrate — the whole estate, both platforms.** You cannot migrate what you cannot see. Enumerate Logic Apps (Azure) **and** Power Automate flows (Power Platform) and record

how each authenticates. A migration program that skips a hidden service-account flow leaves the exact risk it set out to retire.

• **Classify by risk, migrate by risk.** Rank flows by standing-credential risk and blast radius, and migrate the worst first — the shared-service-account flows and the broadest-access flows deliver the most security value earliest.

• **Reference the sibling patterns; don't reinvent per-flow depth.** For each flow, apply the matching product (#1 send/read, #2 change-notification, #3 polling, #5 managed-identity foundation) and scope per #4. The program's job is sequencing, parity, and governance — not re-deriving a managed-identity Graph workflow for every flow.

• **Scope every mailbox permission to only what the flow needs (per #4).** A Graph mail application permission is tenant-wide by default. Constrain it with an Application Access Policy or Graph RBAC-for-Exchange so a migrated flow can reach only its own mailbox(es) — least privilege is the security spine of the whole program.

• **Treat Power Automate honestly.** Power Automate flows typically use connection references, not Azure managed identities, so their modern target may be a re-platform, not a swap. Tag these in classification and plan the extra effort — don't pretend they are identical to Logic Apps.

• **Pilot first, always.** Prove the method on a small representative cohort spanning the sibling patterns, gate it, and only then wave the estate. The pilot exercises every per-flow pattern once before it matters at scale.

• **Prove parity, don't assume it.** Capture each legacy flow's behaviour as a baseline and compare the migrated flow against it. "It runs" is not "it does the same thing" — a silent parity break is the failure mode the baseline catches.

• **Disable legacy flows, delete them last.** Keep every legacy connector flow disabled (not deleted) through a grace period so per-flow rollback stays available; delete only after the migrated replacement passes and the window elapses.

• **Govern the end state.** Run the discovery + validation scan on a schedule to catch a new connector-based flow or a reverted one, and make managed-identity-Graph-first the standard for new automation — so the estate does not re-accumulate the risk.

• **Verify the volatile specifics against Microsoft docs, every time.** Connector managed-identity support, exact Graph permission names, change-notification lifetimes, the Application-Access-Policy-vs-Graph-RBAC choice, and national-cloud endpoints all drift. State the target; confirm the specifics at migration time rather than trusting yesterday's answer.

8. Deliverables Summary — Modernizing Office 365 Connector Workflows to Microsoft Graph

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box

Document	Purpose
1. Solution Overview	Why this is a program not a one-off — the inventory→classify→migrate→govern arc, the per-flow playbook that leans on siblings #1/#2/#3/#5 and defers scoping to #4, benefits, limitations, and the Logic-Apps-vs-Power-Automate reality
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (read access for both platforms, rights to create managed identities + grant/consent Graph permissions, a pilot cohort)
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	The program runbook — inventory, classify, configure pilot targets, pilot cutover, pilot gate, wave expansion, govern
5. Testing Guide	PASS/FAIL gates — inventory completeness, per-flow parity, no residual credentials, scoped permission, rollback, no regression
6. Troubleshooting Guide	Field issues — missed flow, parity break, Power Automate can't take a managed identity, permission/consent gaps, regression
7. Best Practices	Field-proven recommendations for the program

PowerShell scripts

Script	What it does
`O365GRAPH-Discovery.ps1`	Read-only — enumerates Logic Apps using Office 365 connectors and how each authenticates; flags where Power Automate data must be supplied (enumeration method: verify)
`O365GRAPH-Classify.ps1`	Read-only — scores each discovered flow by risk + Graph-migratability and writes the `Migration-Register – Estate – Program.csv`
`O365GRAPH-Validate.ps1`	PASS/FAIL — a migrated flow uses managed identity + Graph and has no stored-credential connection
`O365GRAPH-Report.ps1`	Program dashboard — total connector flows, migrated, remaining, risk buckets + CSV
`O365GRAPH-Set-AppRole.ps1`	The only state-changer — assigns/verifies an application app-role on a migrated flow's managed identity; **defaults to** `-WhatIf`

What the buyer receives

- This program runbook as a **Word** document and **PDF** (cover, table of contents, page numbers, scripts appendix).
- All five **PowerShell scripts** in a single ZIP.
- A repeatable program following the standard model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with a pilot-first gate and a documented per-flow rollback.

Outcome

An Office 365 connector estate migrated at scale onto managed-identity Microsoft Graph workflows — inventoried, risk-ranked, migrated with a repeatable per-flow playbook that leans on the sibling products, parity-validated, and governed so it ends **credential-free** and stays that way.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

O365GRAPH-Classify.ps1

Purpose: Read-only classification for the migration program. Reads the discovery register ("Connector-Flow-Inventory - Estate - Program.csv"), scores every flow on two axes, assigns the sibling migration pattern, and writes the "Migration-Register - Estate - Program.csv". Makes NO changes.

How to run: powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Classify.ps1 `-InputCsv '.\Connector-Flow-Inventory - Estate - Program.csv'` -OutputCsv '.\Migration-Register - Estate - Program.csv' SCOPES / ROLES None beyond read access to the input CSV — this is an offline scoring pass over discovery output. EXPECTED RESULTS The migration register: each flow with a RiskScore, RiskBand (High/Med/Low), Migratability class, assigned SiblingPattern, ScopePerSibling4 flag, and a suggested Wave — sorted so the highest-risk + cleanest flows lead. Tag the pilot cohort from the top of this list. .NOTES Azure Solutions — read-only. Scores discovery output; makes no changes. White-label sample values use contoso.com. "Re-platform" viability is a verify-against-docs item. > [CmdletBinding()] param([Parameter(Mandatory)][string]$InputCsv, [string]$OutputCsv = ".\Migration-Register - Estate - Program.csv")

O365GRAPH-Discovery.ps1

Purpose: Read-only discovery for the "Modernizing Office 365 Connector Workflows to Microsoft Graph" migration program. Enumerates Logic Apps across the in-scope Azure subscriptions, inspects each workflow definition + its API connections, flags those using an Office 365 connector (Office 365 Outlook / Office 365 Users / Excel Online), and records the auth model of each (interactive user, shared service account, stored secret, or an existing managed identity). Writes the "Connector-Flow-Inventory - Estate - Program.csv" register. Makes NO changes.

How to run: powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Discovery.ps1 -SubscriptionId '<sub id>' powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Discovery.ps1 -OutputCsv '.\Connector-Flow-Inventory - Estate - Program.csv' SCOPES / ROLES Azure: Reader across the subscriptions/resource groups holding Logic Apps (Az.Accounts, Az.LogicApp, Az.Resources). Run Connect-AzAccount first, or the script prompts. Power Automate: Power Platform administrator (or read) to enumerate flows (method: verify). EXPECTED RESULTS A CSV register, one row per flow: FlowName, Platform (LogicApp/PowerAutomate), Connector(s), AuthModel, Owner, MailboxesOrResources, ResourceId. A single readiness picture of the whole connector estate and its authentication before you classify and migrate. .NOTES Azure Solutions — read-only. Reports on configuration/identifiers only; makes no changes. White-label sample values use contoso.com. Verify volatile Microsoft specifics against current docs. > [CmdletBinding()] param([string]$SubscriptionId, [string]$OutputCsv = ".\Connector-Flow-Inventory - Estate - Program.csv")

O365GRAPH-Report.ps1

Purpose: Read-only program reporting for the migration program. Reads the migration register ("Migration-Register - Estate - Program.csv") and, optionally, a fresh validation pass, then summarizes program progress: total connector-based flows, how many are migrated (managed identity + Graph, no

stored credential), how many remain, and the risk-band buckets. Prints a dashboard and writes a dated CSV. Makes NO changes.

How to run: powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Report.ps1 ` -Register '.\Migration-Register - Estate - Program.csv' powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Report.ps1 -Register '.\...csv' -OutputCsv '.\program-status.csv' SCOPES / ROLES Read access to the register CSV. (If -LiveValidate is added in your environment, the same Reader / Microsoft.Graph read roles that O365GRAPH-Validate.ps1 uses.) EXPECTED RESULTS A dashboard — Total / Migrated / Remaining, a High/Med/Low risk-band breakdown of what remains, and a per-wave rollup — plus a CSV snapshot for trend tracking across cycles. .NOTES Azure Solutions — read-only. Summarizes register state; makes no changes. White-label sample values use contoso.com. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Register, [string]$OutputCsv = (".\program-status-{0}.csv" -f (Get-Date -Format 'yyyyMMdd')))

O365GRAPH-Set-AppRole.ps1

Purpose: Grants the least-privilege Graph APPLICATION permission a migrated Logic App's managed identity ("<FlowName> - Graph MI - Target") needs to do its job — e.g. a mail permission — by assigning the corresponding app-role from the Microsoft Graph service principal to the managed identity's service principal. If the assignment already exists, it reports it and makes no change (idempotent).

How to run: Dry run (default) — shows the planned assignment, changes nothing: powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Set-AppRole.ps1 `-ManagedIdentityObjectId '<mi sp object id>' -GraphAppRole 'Mail.Read' Apply, after review: powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Set-AppRole.ps1` -ManagedIdentityObjectId '<mi sp object id>' -GraphAppRole 'Mail.Read' -Apply SCOPES / ROLES Microsoft.Graph: AppRoleAssignment.ReadWrite.All + Application.Read.All, and a role able to grant/admin-consent application permissions (e.g. Privileged Role Administrator / Global Administrator). Run Connect-MgGraph with those scopes first. Modules: Microsoft.Graph (Applications, Authentication). EXPECTED RESULTS -WhatIf (default): a printed plan — "WOULD assign app-role '<role>' (id <guid>) on Graph SP to managed identity <name>". -Apply: the app-role is assigned (or reported already present), with the resulting assignment id echoed for the register. .NOTES Azure Solutions — the only state-changer; defaults to -WhatIf. Grants a Graph app-role; scope the mailbox reach per sibling #4. Exact permission NAME is a verify-against-current-docs item. White-label sample values use contoso.com. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$ManagedIdentityObjectId, # the MI's service principal object id [Parameter(Mandatory)][string]$GraphAppRole, # e.g. 'Mail.Read' — VERIFY current least-priv name [string]$GraphAppId = '00000003-0000-0000-c000-000000000000', # Microsoft Graph well-known appId [switch]$Apply)

O365GRAPH-Validate.ps1

Purpose: Read-only validation for the migration program. For a migrated Logic App (or a set), confirms the end-state acceptance criteria and emits PASS/FAIL per flow: 1. The Logic App has a managed identity assigned ("<FlowName> - Graph MI - Target"). 2. The workflow calls Microsoft Graph (not an Office 365 connector). 3. There is NO stored-credential connection (no user token / service-account secret). 4. (Reported) A scoped Graph mail application permission is expected on the identity — the scoping mechanism itself is delivered/verified per sibling #4. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Validate.ps1 -LogicAppName 'Contoso-MailNotify'

powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Validate.ps1 -ResourceGroup 'rg-automation' powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Validate.ps1 -All SCOPES / ROLES Azure: Reader on the Logic App(s) (Az.Accounts, Az.LogicApp). Microsoft.Graph read (Directory.Read.All / Application.Read.All) to confirm the identity's app-role assignments. Run Connect-AzAccount / Connect-MgGraph first, or the script prompts. EXPECTED RESULTS A PASS/FAIL line per flow with the failing criterion named, and a cohort summary (migrated-and-clean vs. still-carrying-a-credential). A migrated flow is PASS only when it uses managed identity + Graph and has zero stored-credential connections. .NOTES Azure Solutions — read-only. Confirms end state; makes no changes. White-label sample values use contoso.com. Exact Graph permission names: verify against current docs. > [CmdletBinding()] param([string]$LogicAppName, [string]$ResourceGroup, [switch]$All)

How to run: powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Validate.ps1 -LogicAppName 'Contoso-MailNotify' powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Validate.ps1 -ResourceGroup 'rg-automation' powershell -ExecutionPolicy Bypass -File .\O365GRAPH-Validate.ps1 -All SCOPES / ROLES Azure: Reader on the Logic App(s) (Az.Accounts, Az.LogicApp). Microsoft.Graph read (Directory.Read.All / Application.Read.All) to confirm the identity's app-role assignments. Run Connect-AzAccount / Connect-MgGraph first, or the script prompts. EXPECTED RESULTS A PASS/FAIL line per flow with the failing criterion named, and a cohort summary (migrated-and-clean vs. still-carrying-a-credential). A migrated flow is PASS only when it uses managed identity + Graph and has zero stored-credential connections. .NOTES Azure Solutions — read-only. Confirms end state; makes no changes. White-label sample values use contoso.com. Exact Graph permission names: verify against current docs. > [CmdletBinding()] param([string]$LogicAppName, [string]$ResourceGroup, [switch]$All)